

Evaluation Measures

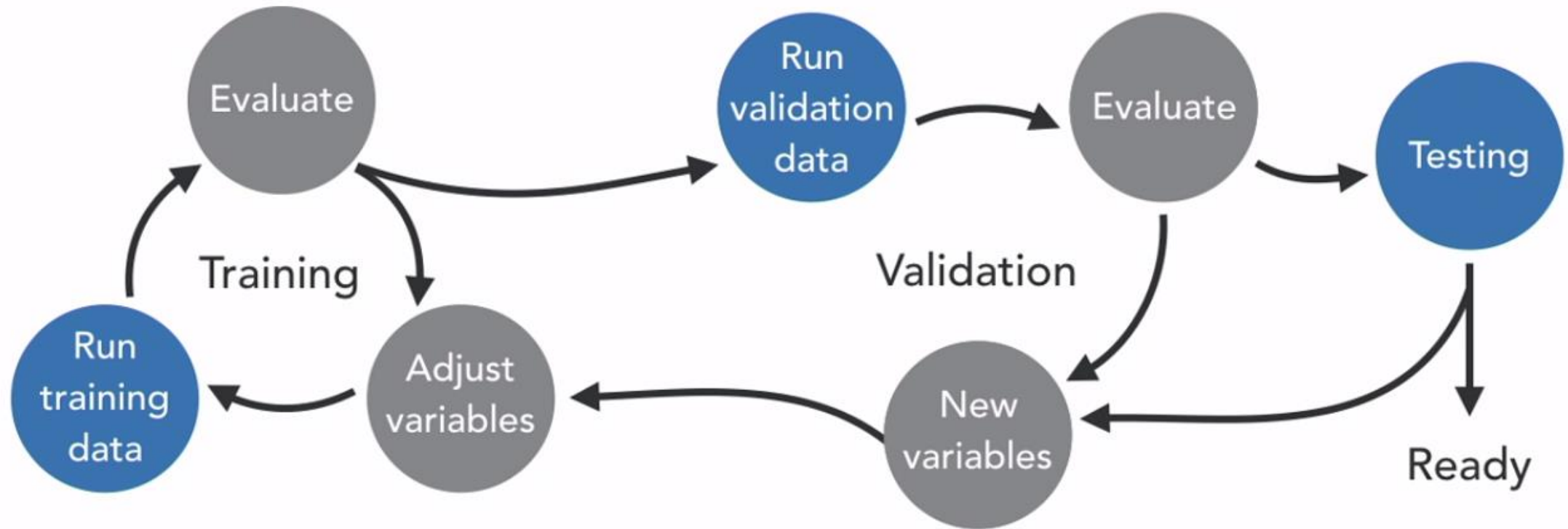
**how to
evaluate
my model?**



How Do We Split Up Our Data?

- Training dataset – data used to train the model (allow the algorithm to learn from this data)
- Validation dataset – data used to select the best model (optimal algorithm and hyperparameter settings)
- Test dataset – data used to provide an unbiased evaluation of what the model will look like in its real environment

What Does This Look Like?



What Are the Risks of Not Splitting Up the Full Dataset?

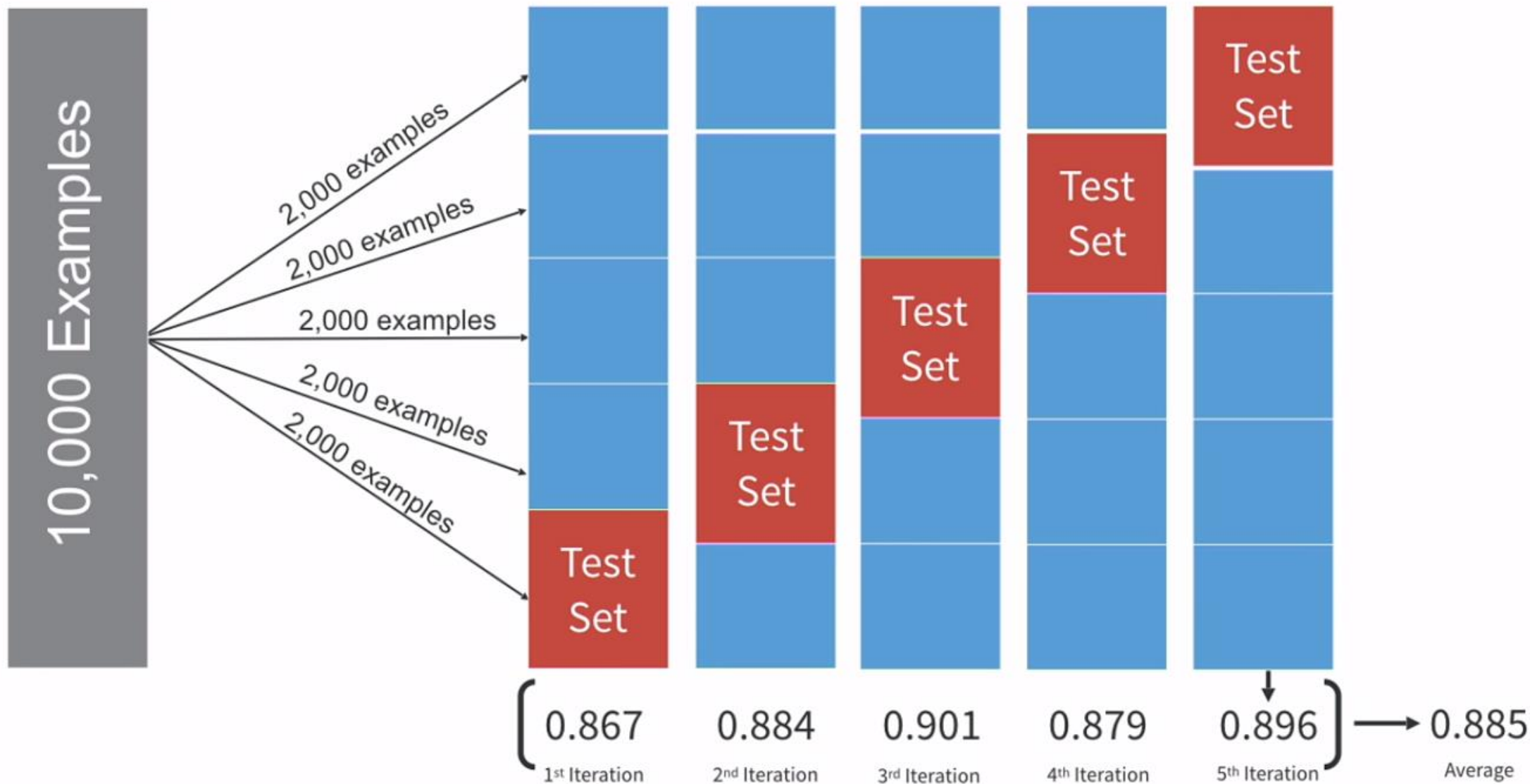
- Overfitting or underfitting to the data
- Inaccurate representation of how the model will generalize

Holdout Test Set

Sample of data not used in fitting a model; used to evaluate the model's ability to generalize to unseen data

K-Fold Cross-Validation

Data is divided into k subsets and the holdout method is repeated k times. Each time, one of the k subsets is used as the test set and the other $k-1$ subsets are combined to be used to train the model.



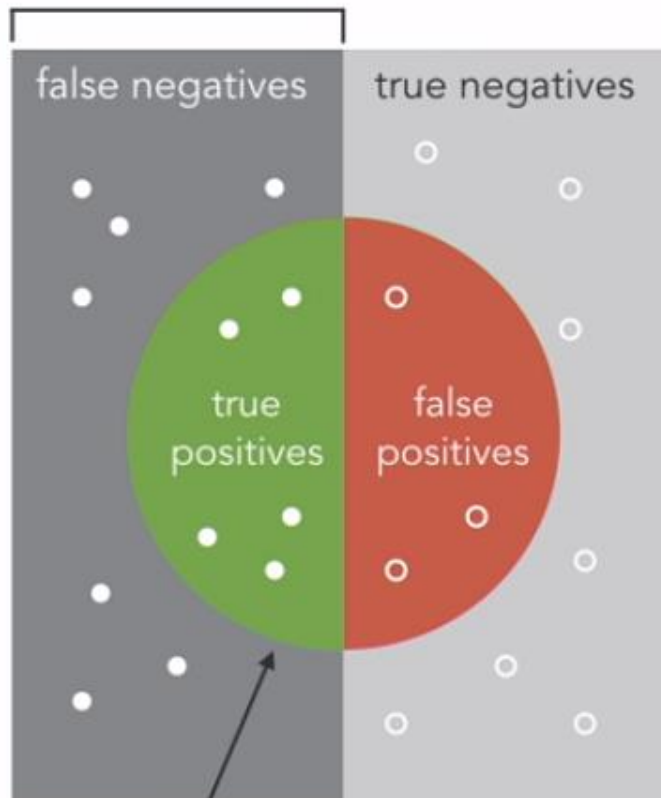
Two Components of Evaluation Framework

1. Evaluation metrics – how are we gauging the accuracy of the model?
2. Process (how to split the data) – how do we leverage our full dataset to mitigate likelihood of overfitting or underfitting?

Evaluation Metrics

- Accuracy = $\frac{\text{\# predicted correctly}}{\text{total \# of examples}}$
- Precision = $\frac{\text{\# predicted as surviving that actually survived}}{\text{total \# predicted to survive}}$
- Recall = $\frac{\text{\# predicted as surviving that actually survived}}{\text{total \# that actually survived}}$

relevant elements



selected elements

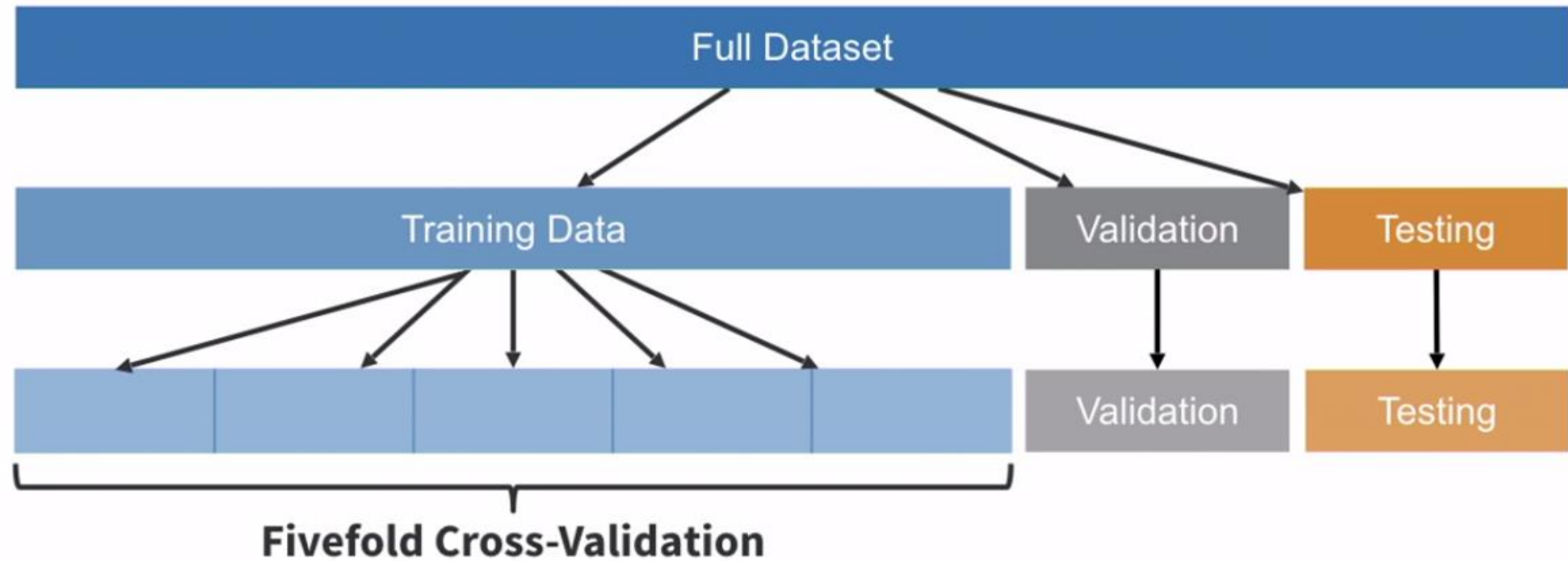
How many selected items are relevant?

$$\text{Precision} = \frac{\text{true positives}}{\text{true positives} + \text{false positives}}$$

How many selected items are relevant?

$$\text{Recall} = \frac{\text{true positives}}{\text{true positives} + \text{false negatives}}$$

What Does This Look Like?



Process

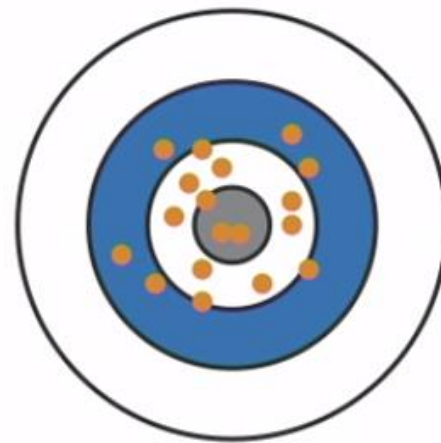
1. Run fivefold cross-validation and select the best models.
2. Re-fit models on full training set, evaluate those models on the validation set and pick the best one.
3. Evaluate that best model on the test set to gauge its ability to generalize to unseen data.

Bias-Variance Trade-off

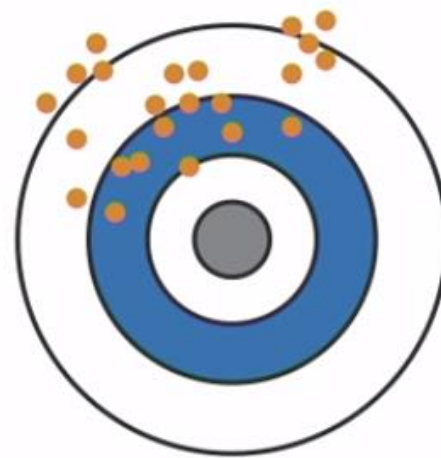
Low Variance

High Variance

Low Bias

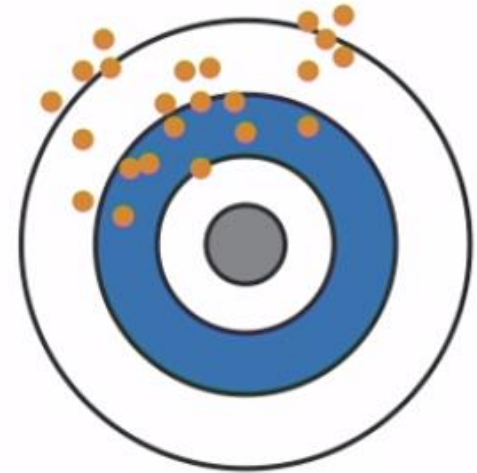


High Bias



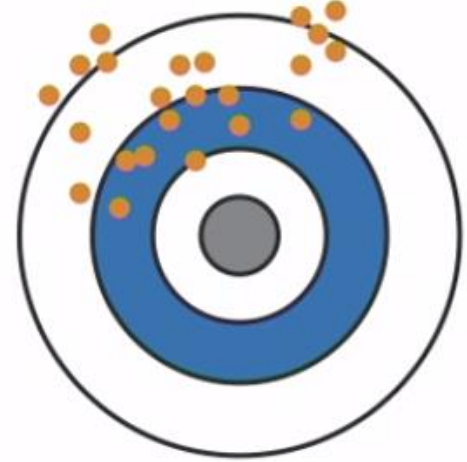
Bias is the algorithm's tendency to consistently learn the wrong thing by not taking into account all the information in the data.

High bias is a result of the algorithm missing the relevant relations between features and target outputs.

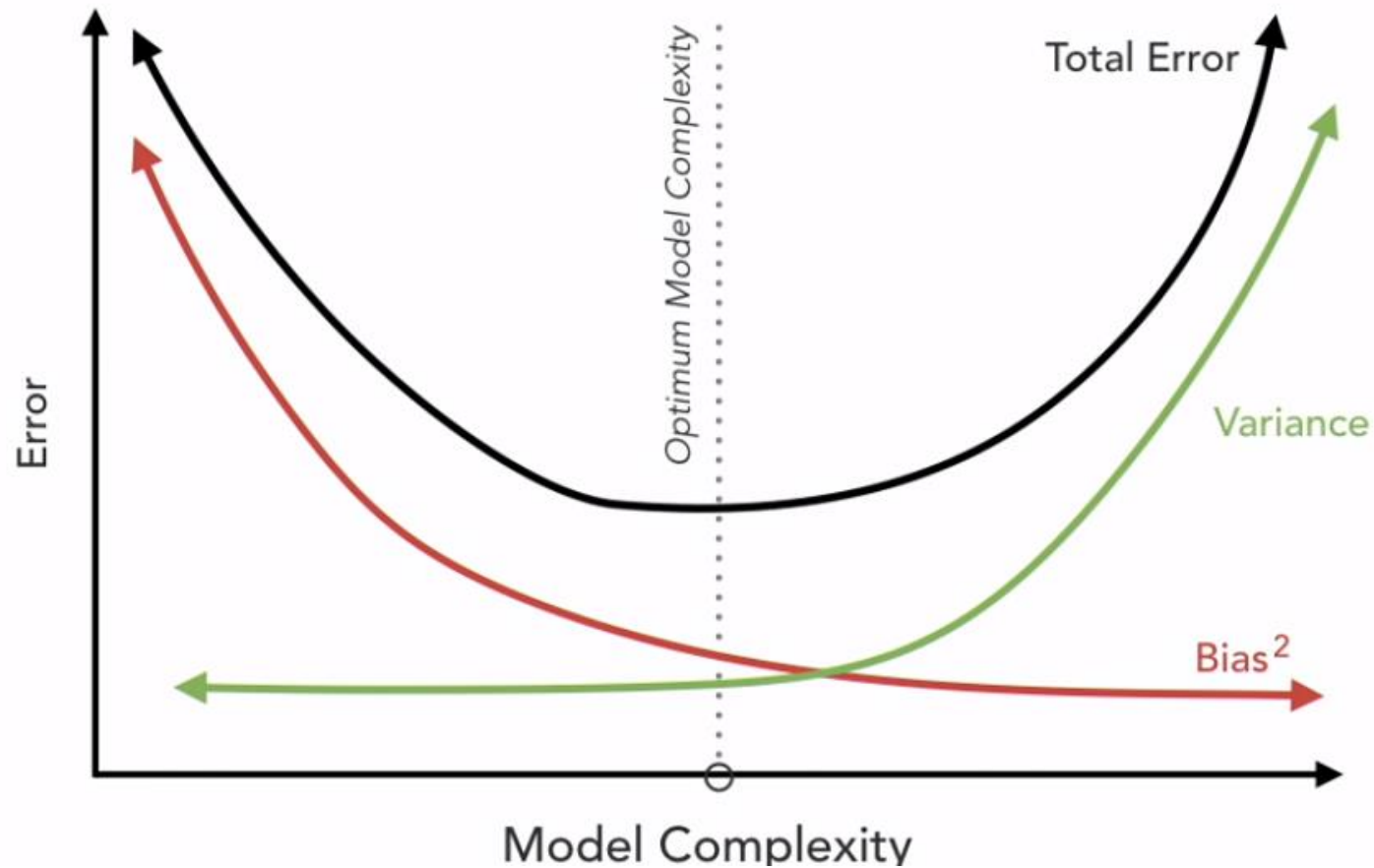


Variance refers to an algorithm's sensitivity to small fluctuations in the training set.

High variance is a result of the algorithm fitting to random noise in the training data.

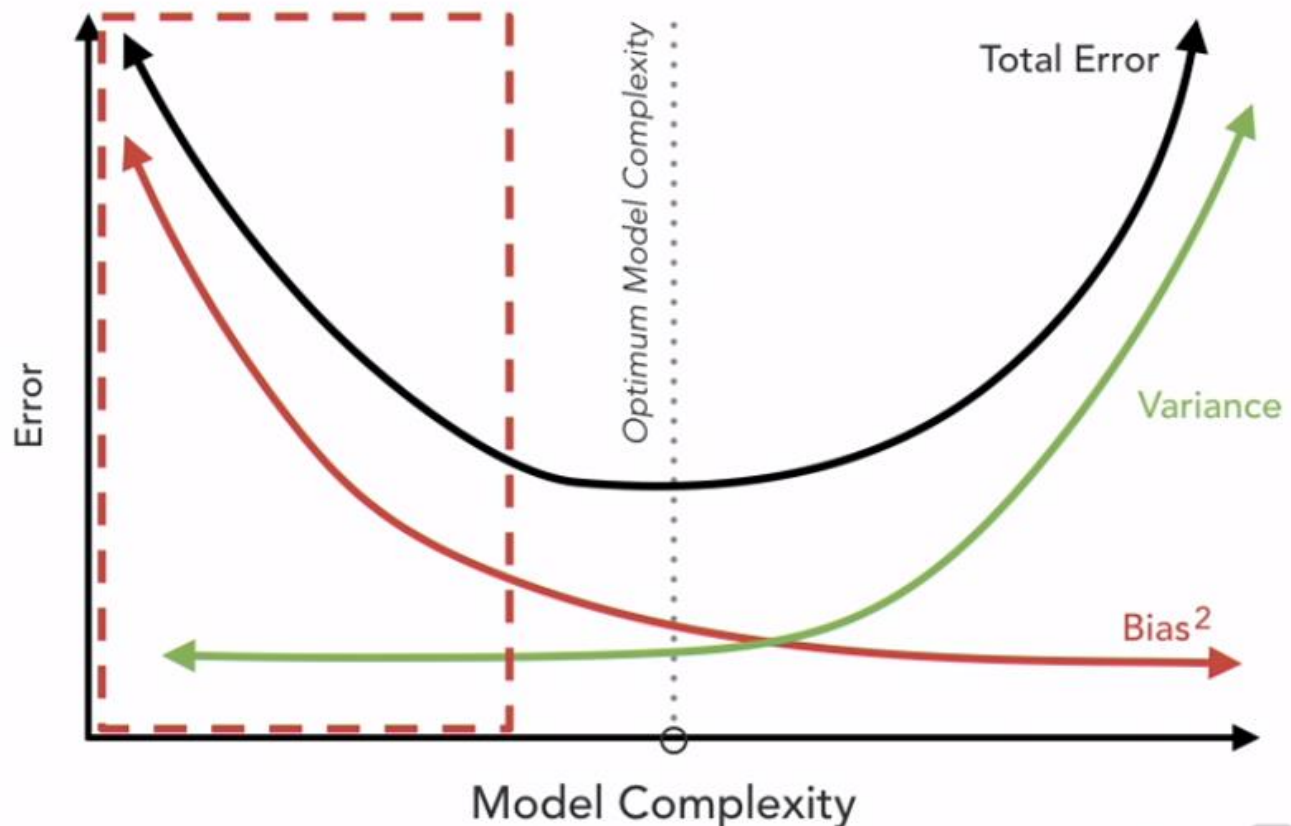


$$\text{Total Error} = (\text{Bias} + \text{Variance}) + \text{Irreducible Error}$$



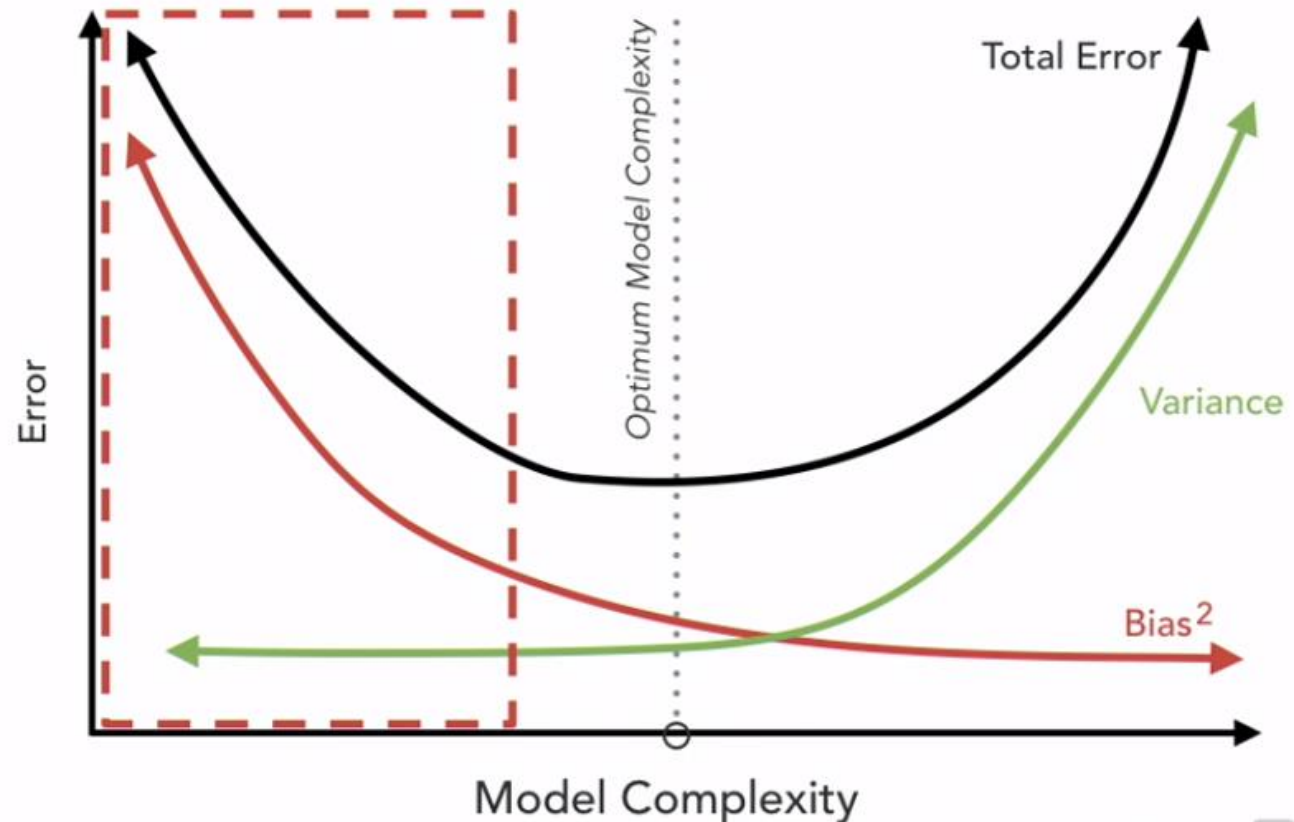
$$\text{Total Error} = (\text{Bias} + \text{Variance}) + \text{Irreducible Error}$$

Underfitting occurs when an algorithm cannot capture the underlying trend of the data.

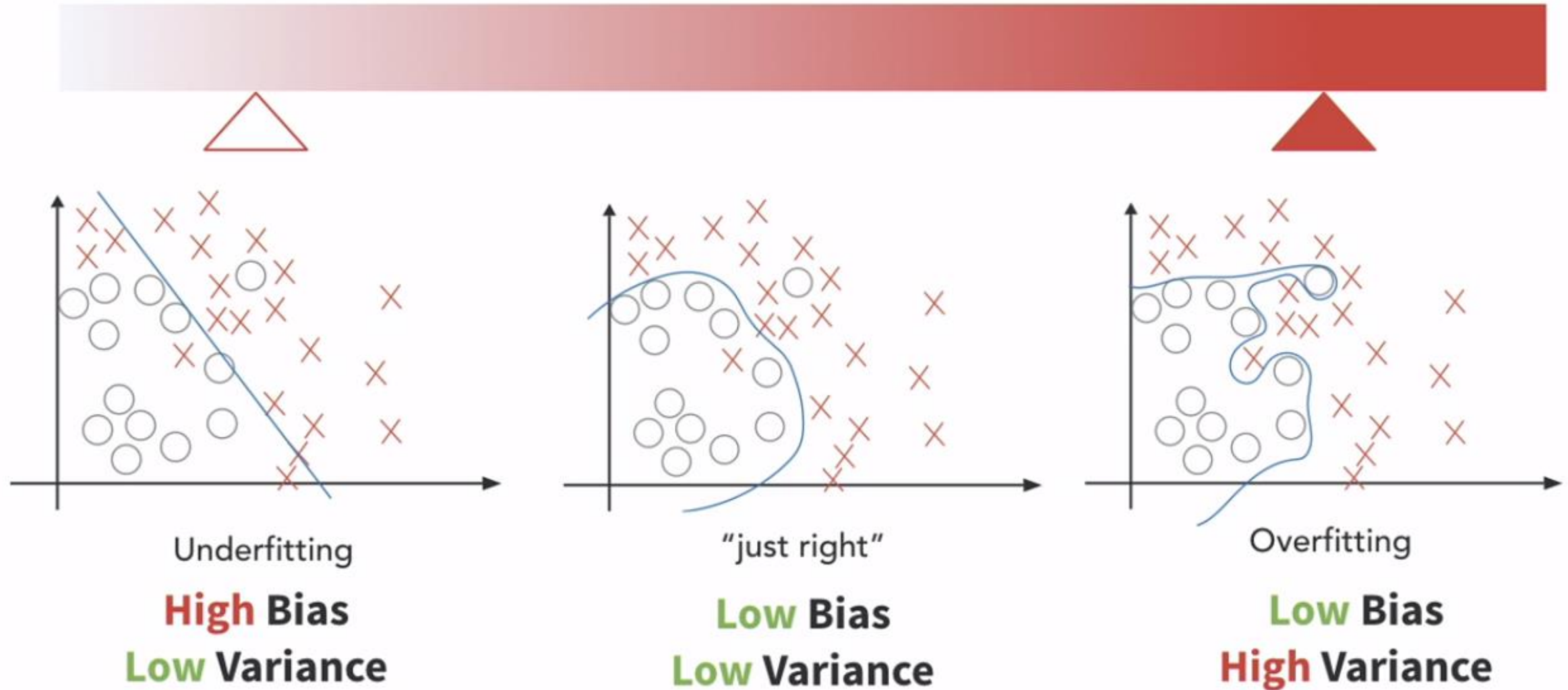


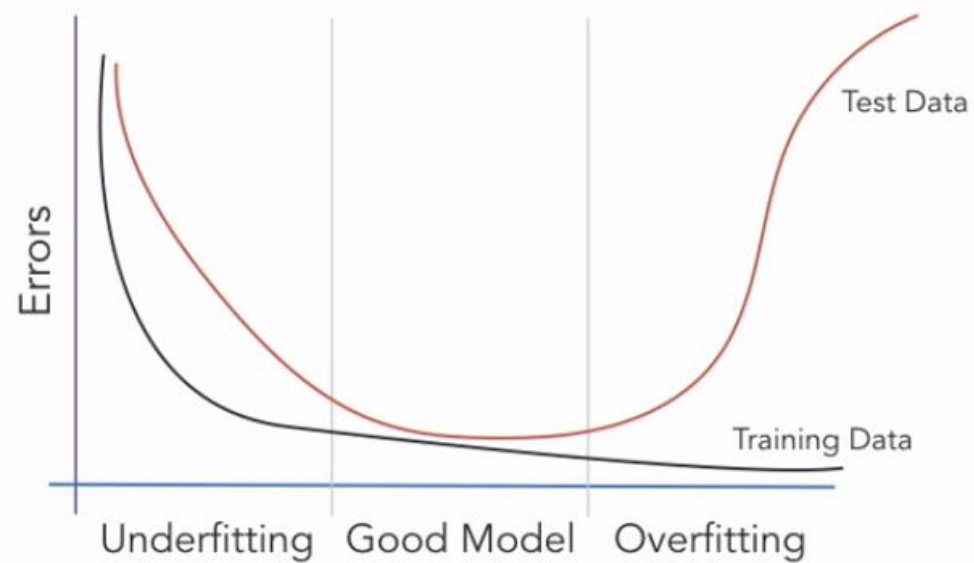
$$\text{Total Error} = (\text{Bias} + \text{Variance}) + \text{Irreducible Error}$$

Overfitting occurs when an algorithm fits too closely to a limited set of data.



Complexity





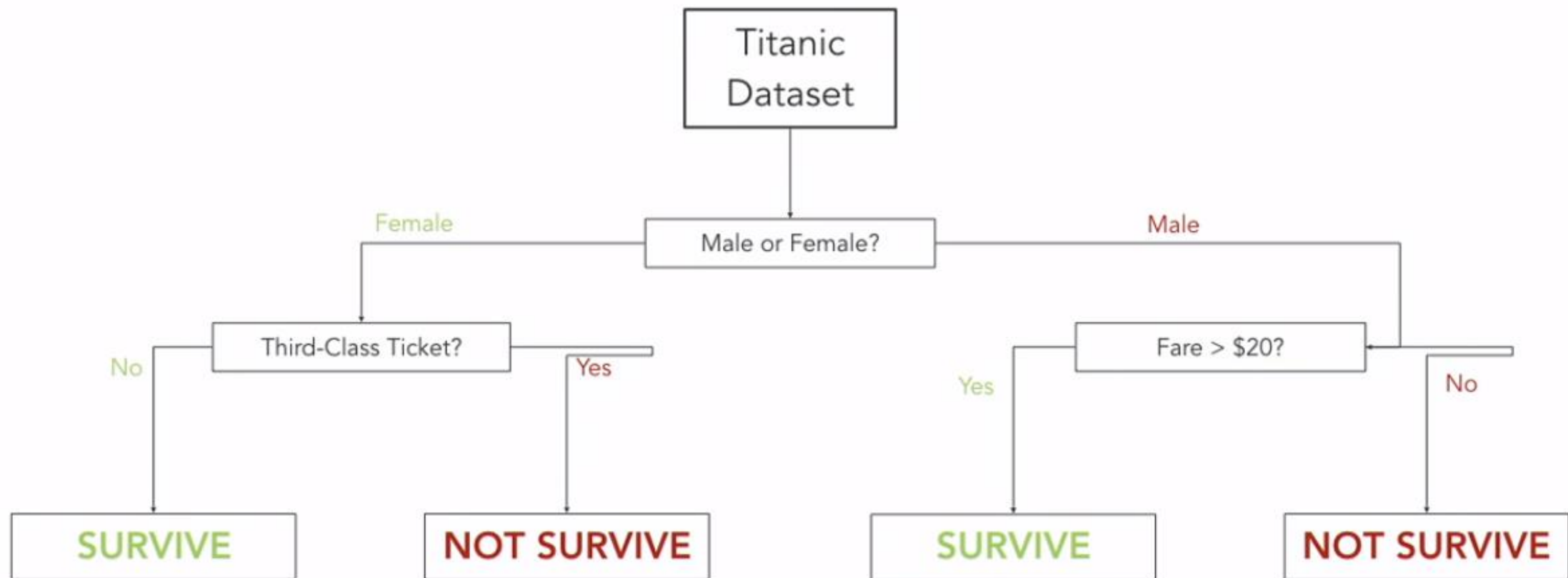
	Underfitting	Good Model	Overfitting
Complexity	Low	Medium	High
Bias	High	Low	Low
Variance	Low	Low	High
Train Error	High	Low	Low
Test Error	High	Low	High

Two Primary Ways to Tune Model for Optimal Complexity

1. Hyperparameter tuning – choosing a set of optimal hyperparameters for fitting an algorithm
2. Regularization – technique used to reduce overfitting by discouraging overly complex models in some way

A model **parameter** is a configuration variable that is internal to the model and whose value can be estimated from data.

A model **hyperparameter** is a configuration that is external to the model, whose value cannot be estimated from data, and whose value guides how the algorithm learns parameter values from the data.



Parameters – fare = \$20, ticket class = third

Hyperparameters – max depth of tree, features to consider, etc.

Two Primary Ways to Tune Model for Optimal Complexity

1. ~~Hyperparameter tuning – choosing a set of optimal hyperparameters for fitting an algorithm~~
2. Regularization – technique used to reduce overfitting by discouraging overly complex models in some way

Goal – allow enough flexibility for the algorithm to learn the underlying patterns in the data but provide guardrails so it doesn't overfit

Grid Search Plotting

- When it comes to hyperparameter tuning, most people use grid search.
- Grid search is a brute force mechanism for trying all possible combinations of the algorithm's parameters.
- By establishing a range of discrete values for multiple hyperparameters, each can be seen as an axis, and the grid is just the set of points representing every combination.
- Each combination is tested individually and the best is returned.

Examples of Regularization

1. Ridge regression and lasso regression – adding a penalty to the loss function to constrain coefficients
2. Dropout – some nodes are ignored during training which forces the other nodes to take on more or less responsibility for the input/output

Metrics for Regression

Metric	Measures	In Scikit-learn
Mean Square Error (MSE, RMSE)	distance between predicted values and actual values (more sensitive to outliers)	<pre>from sklearn.metrics import mean_squared_error</pre>
Absolute Error (MAE, RAE)	distance between predicted values and actual values (less sensitive to outliers)	<pre>from sklearn.metrics import mean_absolute_error, median_absolute_error</pre>
Coefficient of Determination (R-Squared)	% of variance explained by the regression; how well future samples are likely to be predicted by the model	<pre>from sklearn.metrics import r2_score</pre>

Metrics for Classification

Metric	Measures	In Scikit-learn
Precision	How many selected are relevant?	<code>from sklearn.metrics import precision_score</code>
Recall	How many relevant were selected?	<code>from sklearn.metrics import recall_score</code>
F1	Weighted average of precision & recall	<code>from sklearn.metrics import f1_score</code>
Confusion Matrix	True positives, true negatives, false positives, false negatives	<code>from sklearn.metrics import confusion_matrix</code>
ROC	True positive rate vs. false positive rate, as classification threshold varies	<code>from sklearn.metrics import roc</code>
AUC	Aggregate accuracy, as classification threshold varies	<code>from sklearn.metrics import auc</code>

Confusion Matrices

		predicted	
		p'	n'
actual	p	true positives	false negatives ← Type II error
	n	false positives ↑ Type I error	true negatives